

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



BÀI TẬP LỚN
MẠNG MÁY TÍNH (TN) - CO3094

**DEVELOP A NETWORK
APPLICATION**

Lớp: L01 - HK242

Giảng viên hướng dẫn: Bùi Xuân Giang

Stt	Họ và tên	MSSV
1	Đặng Quốc An	2210002
2	Nguyễn Phúc An	2210022
3	Ngô Trường Bách	2210183
4	Nguyễn Duy Bách	2212185

Thành phố Hồ Chí Minh, tháng 04 năm 2025

Mục lục

Danh sách hình vẽ	3
1 Mở đầu	4
1.1 Giới thiệu	4
1.2 Đối tượng nghiên cứu	4
1.3 Mục tiêu nghiên cứu	4
2 Nội dung	5
2.1 System Infrastructure in Hybrid Paradigm	5
2.1.1 Lý thuyết	5
2.1.2 Hiện thực hóa hệ thống	6
2.2 Authentication	8
2.2.1 Lý thuyết xác thực người dùng	8
2.2.2 Hiện thực chức năng xác thực	9
2.3 Channel và Channel Hosting	11
2.3.1 Lý thuyết hệ thống kênh (Channel Theory)	11
2.4 Hiện thực hệ thống kênh	14
2.5 User Access	16
2.5.1 Lý thuyết	16
2.5.2 Hiện thực hệ thống	17
2.6 System Log	18
2.6.1 Lý thuyết ghi log hệ thống	18
2.6.2 Hiện thực ghi log trong hệ thống	19
2.7 Synchronization giữa Channel-hosting và Server trung tâm	21
2.7.1 Lý thuyết đồng bộ dữ liệu	21
2.7.2 Tình huống và chiến lược đồng bộ	21
2.7.3 Hiện thực kỹ thuật trong mã nguồn	22
2.8 Livestream Functionality	23
2.8.1 Lý thuyết truyền hình ảnh thời gian thực qua mạng P2P	23

2.8.2	Các công cụ và thư viện sử dụng	24
2.8.3	Hiện thực quá trình livestream	24
3	Các bước thực hiện hệ thống (System Workflows)	26
3.1	Tình huống 1: Đăng nhập hoặc truy cập với tư cách khách	26
3.2	Tình huống 2: Tạo một kênh mới	26
3.3	Tình huống 3: Tham gia một channel đã tồn tại	27
3.4	Tình huống 4: Gửi tin nhắn trong channel	27
3.5	Tình huống 5: Bắt đầu và xem livestream	28
3.6	Tình huống 6: Người dùng trở lại sau khi offline	28
3.7	Tình huống 7: Người dùng đăng xuất và offline	29
4	Nguồn dữ liệu và nguồn code	30



Danh sách hình vẽ

1	Sơ đồ minh họa cho bài tập lớn	6
---	--	---

1 Mở đầu

1.1 Giới thiệu

Trong thời đại số hóa mạnh mẽ, nhu cầu về các nền tảng giao tiếp tức thời, linh hoạt và hiệu quả ngày càng trở nên cấp thiết, đặc biệt trong bối cảnh làm việc nhóm và học tập từ xa. Ứng dụng trò chuyện thời gian thực (chat application) không chỉ đơn thuần là công cụ giao tiếp, mà còn là nền tảng hỗ trợ tổ chức, cộng tác, chia sẻ dữ liệu và phát trực tiếp (livestream).

Trong bài tập lớn môn Mạng Máy Tính này, nhóm xây dựng một ứng dụng mạng phân đoạn chat đơn giản, theo mô hình kết hợp Client-Server và Peer-to-Peer (Hybrid Paradigm). Ứng dụng hướng đến khả năng mở rộng, phân tải hiệu quả và hỗ trợ các tính năng thiết yếu như xác thực người dùng, tạo/duyệt kênh, nhắn tin và truyền phát dữ liệu trực tiếp.

1.2 Đối tượng nghiên cứu

Bài tập lớn này tập trung vào việc xây dựng và mô phỏng một ứng dụng chat đơn giản, với các thành phần chính bao gồm kiến trúc mạng lai (hybrid paradigm), hệ thống tracker và giao thức khởi tạo kết nối và giao diện người dùng.

1.3 Mục tiêu nghiên cứu

Mục tiêu của bài tập là giúp sinh viên hiểu các nguyên lý cơ bản của 1 ứng dụng chat cơ bản thông qua:

- Xây dựng một ứng dụng mạng theo kiến trúc kết hợp Client-Server và P2P.
- Thiết kế và hiện thực đầy đủ giao thức giao tiếp.
- Đảm bảo tính năng bảo mật cơ bản và phân quyền người dùng.

2 Nội dung

2.1 System Infrastructure in Hybrid Paradigm

2.1.1 Lý thuyết

Trong mô hình mạng lai (hybrid network paradigm), hệ thống được kết hợp giữa hai kiến trúc phổ biến: Client-Server và Peer-to-Peer (P2P), nhằm tận dụng thế mạnh của cả hai:

- **Client-Server:** phù hợp cho giai đoạn khởi tạo kết nối, lưu trữ backup, và cung cấp dịch vụ đồng bộ tập trung. Tại đây, server trung tâm đóng vai trò tracker, chịu trách nhiệm duy trì danh sách các peer đang kết nối và hỗ trợ điều phối quá trình kết nối.
- **P2P:** cho phép các peer truyền trực tiếp nội dung cho nhau, giảm tải cho server trung tâm, tối ưu hiệu suất truyền dữ liệu theo thời gian thực (livestream, tin nhắn chat).

Các thành phần chính trong mô hình:

- **Peer:** là một instance ứng dụng độc lập. Một máy có thể chạy nhiều peer bằng cách khởi tạo nhiều cổng khác nhau.
- **Tracker Server:** chạy ở máy trung tâm, chịu trách nhiệm lưu trữ danh sách các peer đang online, xử lý yêu cầu khởi tạo và phân phối thông tin peer list.

Quy trình kết nối:

1. **submit_info:** Peer mới gửi thông tin IP và cổng tới tracker server.
2. **add_list:** Tracker lưu thông tin peer vào danh sách theo dõi.
3. **get_list:** Peer gửi yêu cầu và nhận danh sách peer hiện tại từ tracker.
4. **peer_connect:** Peer sử dụng danh sách để kết nối trực tiếp với các peer khác.

System infrastructure in hybrid paradigm

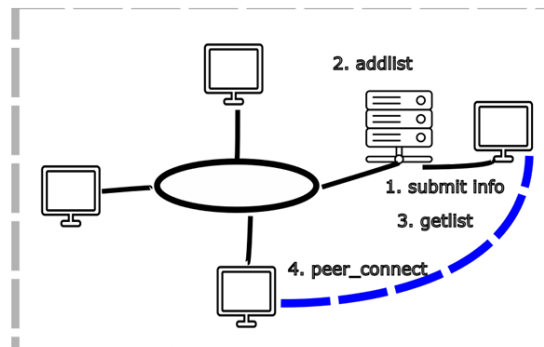


Figure 1. Illustration of segment chat system.

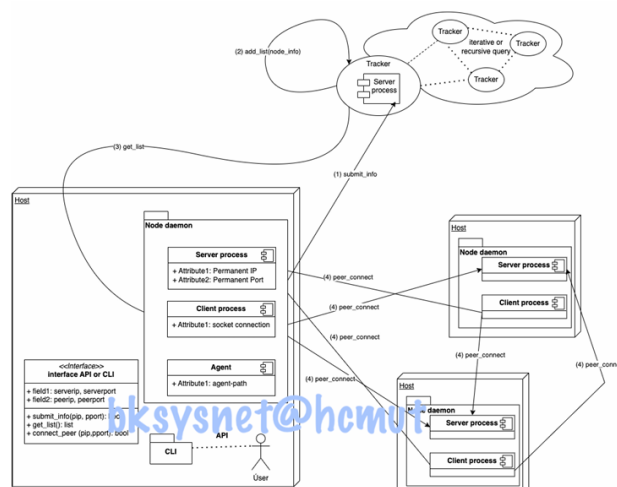


Figure 2 Updated design schematic with agency and multi (server) tracker

Hình 1: Sơ đồ minh họa cho bài tập lớn

2.1.2 Hiện thực hóa hệ thống

Các tệp mã nguồn chính

- `tracker.py`: quản lý server trung tâm, xử lý xác thực, quản lý các yêu cầu từ phía client, xử lý các kết nối và danh sách peer.
- `user.py`: thực thi chức năng của một peer như kết nối tracker, gửi/nhận dữ liệu P2P.
- `Run.py`: giao diện chính để khởi động tracker, peer, và UI.

Các hàm và đoạn mã hiện thực

2.1.2.1 *submit_info* – file *user.py*

Peer mới gửi thông tin kết nối của mình tới tracker:

Listing 1: Hàm gửi thông tin peer đến tracker

```
1 request = json.dumps({
2     "command": "CONNECT",
3     "name": self.name,
4     "ip": self.ip,
5     "port": self.port
6 })
7 self.tracker_socket.sendall(request.encode('utf-8'))
```

2.1.2.2 *add_list* – file *tracker.py*

Tracker xử lý và thêm peer vào danh sách:

Listing 2: Thêm peer vào danh sách theo dõi

```
1 self.peer_list.append((name, ip, port, conn))
```

2.1.2.3 *get_list* – file *user.py*

Peer yêu cầu danh sách các peer hiện có từ tracker:

Listing 3: Gửi yêu cầu lấy peer list

```
1 request = json.dumps({"command": "GET_LIST", "name": self.name})
2 response = self.tracker_socket.recv(1024).decode('utf-8')
```

2.1.2.4 *peer_connect* – file *user.py*

Peer kết nối trực tiếp đến các peer khác và gửi tin nhắn:

Listing 4: Gửi tin nhắn tới peer mục tiêu

```
1 s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

```
2 s.connect((target_ip, target_port))  
3 s.sendall(message_data.encode('utf-8'))
```

Toàn bộ luồng kết nối được thiết kế để đảm bảo tính mở rộng và giảm thiểu độ trễ, đặc biệt trong các tình huống truyền tải dữ liệu theo thời gian thực như livestream.

2.2 Authentication

2.2.1 Lý thuyết xác thực người dùng

Hệ thống hỗ trợ hai chế độ truy cập khác nhau cho người dùng: **Visitor mode** và **Authenticated-user mode**, nhằm đảm bảo tính linh hoạt trong kiểm soát truy cập và bảo mật nội dung.

Visitor Mode

Chế độ khách truy cập (visitor) cho phép người dùng truy cập nội dung của hệ thống mà không cần đăng nhập. Tuy nhiên, quyền truy cập này bị giới hạn:

- Không yêu cầu xác thực, người dùng chỉ cần khai báo tên để vào hệ thống.
- Chỉ được phép xem nội dung các kênh, không được phép tạo mới hoặc chỉnh sửa.
- Nếu kênh được cấu hình không cho phép khách truy cập, họ sẽ bị từ chối quyền xem.

Authenticated-user Mode

Chế độ người dùng xác thực yêu cầu người dùng phải đăng nhập để truy cập và sử dụng toàn bộ tính năng hệ thống:

- Bắt buộc phải đăng nhập bằng tài khoản hợp lệ.
- Được phép tạo, chỉnh sửa nội dung, tham gia kênh, livestream,...
- Trạng thái hiển thị được phân thành:

- **Online**: hiển thị đang hoạt động cho các người dùng khác.
- **Offline**: không kết nối hệ thống, không hiển thị.
- **Invisible**: ẩn trạng thái, nhưng vẫn hoạt động ngầm như chế độ online.

2.2.2 Hiện thực chức năng xác thực

Các tệp mã nguồn liên quan

- **Login.py**: giao diện đăng nhập, đăng ký, hoặc truy cập với tư cách khách.
- **authService.py**: xử lý logic đăng ký, đăng nhập, cập nhật trạng thái người dùng.
- **authRequest.py**, **authController.py**: định tuyến request và chuyển tiếp xử lý.
- **Home.py**: giao diện hiển thị nội dung kênh và trạng thái người dùng.

Xử lý Visitor Mode

Trong giao diện, người dùng có thể nhấn nút **Login as Viewer**. Khi đó, hệ thống khởi tạo một phiên không cần xác thực:

Listing 5: Đăng nhập chế độ Viewer (Login.py)

```
1 def login_as_viewer(self):
2     username = self.username_input.text().strip() or "Viewer"
3     self.username = username
4     self.session_info = {"session_id": "dummy-session-id"}
5     self.viewer_login_success.emit(self.username, self.session_info)
```

Giao diện được khóa các nút gửi tin nhắn hoặc tạo kênh:

Listing 6: Vô hiệu hóa quyền ở chế độ Viewer (Run.py)

```
1 self.main_window.send_button.setEnabled(False)
2 self.main_window.message_input.setEnabled(False)
3 self.main_window.add_channel_button.setEnabled(False)
```

Xử lý Authenticated-user Mode

Người dùng đăng nhập thông qua form điền tên và mật khẩu. Request được gửi đến server:

Listing 7: Xử lý đăng nhập (Login.py)

```
1 request_data = {
2     "action": "login",
3     "username": username,
4     "password": password
5 }
6 self.client_socket.send(json.dumps(request_data).encode())
```

Ở phía server (authService.py), thông tin người dùng được kiểm tra và cập nhật phiên làm việc:

Listing 8: Ghi nhận phiên đăng nhập

```
1 new_session = {
2     "peer_ip": peer_ip,
3     "peer_port": peer_port,
4     "session_id": session_id,
5     "login_time": datetime.now(timezone.utc).isoformat(),
6     "visible": True
7 }
8 users_collection.update_one(
9     {"username": user.username},
10    {"$push": {"sessions": new_session}}
11 )
```

Cập nhật trạng thái người dùng

Người dùng có thể chọn trạng thái từ giao diện (Online / Invisible / Offline). Khi thay đổi, hệ thống gửi request cập nhật:

Listing 9: Thay đổi trạng thái (Home.py)

```
1 if status == "Online":
```

```
2     request_data = {"action": "update_status", "session_id": session_id, "
    visible": True}
3 elif status == "Invisible":
4     request_data = {"action": "update_status", "session_id": session_id, "
    visible": False}
5 elif status == "Offline":
6     request_data = {"action": "logout", "session_id": session_id}
```

Server ghi nhận lại trạng thái này để điều chỉnh cách hiển thị người dùng trong danh sách:

Listing 10: Ghi nhận trạng thái người dùng

```
1 users_collection.update_one(
2     {"_id": user["_id"], "sessions.session_id": session_id},
3     {"$set": {"sessions.$.visible": visible}}
4 )
```

Chức năng xác thực người dùng không chỉ đảm bảo tính bảo mật cơ bản mà còn hỗ trợ phân quyền hợp lý và tối ưu trải nghiệm sử dụng hệ thống.

2.3 Channel và Channel Hosting

2.3.1 Lý thuyết hệ thống kênh (Channel Theory)

Trong hệ thống này, **Channel** là đơn vị cơ bản lưu trữ và hiển thị nội dung tin nhắn giữa các người dùng. Mỗi channel là một danh sách các tin nhắn được đồng bộ giữa các peer và/hoặc server trung tâm. Các đặc điểm chính bao gồm:

- Mỗi người dùng có thể **liệt kê danh sách các channel đã tham gia**.
- Mỗi channel có cửa sổ hiển thị tin nhắn dạng *scroll view* và ô nhập liệu.
- Người dùng có thể gửi tin nhắn mới nhưng không thể sửa/xoá (do không nằm trong phạm vi của môn học).
- Việc gửi tin nhắn sẽ kích hoạt cơ chế thông báo đến các peer khác.

- Các kênh có thể tùy chỉnh quyền truy cập, tuy nhiên đây là tính năng tùy chọn (không tính điểm).

Khái niệm Channel Hosting

Trong hệ thống, khái niệm **Server** không dùng để chỉ máy vật lý mà để chỉ peer đang lưu trữ một tập hợp các channel – gọi là **Channel Hosting**. Các điểm chính:

- **Hosted Channels:** Là các kênh do người dùng tạo, được lưu trữ trên peer của họ. Có thể sao lưu tại server trung tâm.
- **Joined Channels:** Là các kênh người dùng tham gia nhưng không sở hữu, dữ liệu được lưu tạm thời (cache).
- Trong trạng thái online, **channel hosting** và **server trung tâm** được đồng bộ **2 chiều**.
- Khi livestream diễn ra, nội dung được truyền trực tiếp theo mô hình P2P.
- Server trung tâm chỉ được truy cập khi peer hosting offline và lưu lượng nhỏ.

Kịch bản đồng bộ dữ liệu giữa các thành phần

Dưới đây là mô tả quá trình đồng bộ giữa peer hosting, server trung tâm, và các peer còn lại trong từng trạng thái cụ thể:

2.3.1.1 Channel hosting offline

- Server chứa bản sao dữ liệu A.
- Người dùng X và Y fetch dữ liệu từ server.
- Nếu có nội dung mới B:
 - X/Y tạo tin mới B → gửi lên server.
 - Nếu peer hosting online trở lại → sync cả A và B từ server.

2.3.1.2 *Channel hosting online*

- Dữ liệu A được sync từ peer hosting lên server.
- Người dùng khác fetch trực tiếp từ peer hosting.
- Nội dung mới B được sync song song về cả hai.

2.3.1.3 *Livestream period*

- Nội dung A được gửi trực tiếp từ hosting đến các peer theo mô hình P2P.
- Một bản sao A có thể được lưu trên server trung tâm (phòng khi hosting offline).

2.3.1.4 *Joined user online*

- Nếu peer hosting online $\Rightarrow$ truy cập trực tiếp.
- Nếu offline $\Rightarrow$ fetch từ server trung tâm.
- Nếu người dùng có nội dung B khi offline:
 - Đồng bộ trở lại channel hosting và server khi online.

2.3.1.5 *5. Joined user offline*

- Không có hành động đồng bộ nào diễn ra.
- Nội dung được cache cục bộ và đồng bộ sau khi online trở lại.

Lưu ý: Chỉ người dùng đã xác thực mới có quyền tạo nội dung mới. Khách truy cập (visitor) chỉ được xem, không được tạo hay chỉnh sửa dữ liệu.

2.4 Hiện thực hệ thống kênh

Các tệp mã nguồn chính

- `channelService.py`, `channelModel.py`: định nghĩa logic xử lý kênh, quyền truy cập, danh sách thành viên, join request.
- `channelRequest.py`, `channelController.py`: định tuyến và gọi dịch vụ tương ứng.
- `Home.py`: giao diện người dùng, hiển thị kênh, gửi tin nhắn, chấp nhận yêu cầu, livestream.
- `syncService.py`: thực hiện đồng bộ kênh giữa peer hosting và server.

Tạo channel

Người dùng xác thực có thể tạo channel từ giao diện:

Listing 11: Tạo kênh mới (`Home.py`)

```
1 request_data = {  
2     "action": "create_channel",  
3     "host": self.username,  
4     "channel_name": new_channel,  
5     "is_private": is_private,  
6     "allow_visitor": allow_visitor  
7 }
```

Xử lý tại server:

Listing 12: Thêm kênh vào MongoDB (`channelService.py`)

```
1 new_channel = Channel(  
2     channel_name=channel_name,  
3     owner=host,  
4     members=[host],  
5     is_private=is_private,  
6     join_requests=[]
```

```
7 )  
8 channels_collection.insert_one(new_channel.dict())
```

Gửi tin nhắn và đồng bộ

Người dùng gửi tin nhắn trong channel:

Listing 13: Gửi tin nhắn (channelService.py)

```
1 new_message = {"sender": username, "text": message_text}  
2 channels_collection.update_one(  
3     {"channel_name": channel_name},  
4     {"$push": {"messages": new_message}}  
5 )
```

Các tin nhắn được hiển thị ở giao diện và cập nhật liên tục:

Listing 14: Giao diện hiển thị tin nhắn (Home.py)

```
1 self.chat_display.append(f"{sender}: {text}")
```

Đồng bộ nội dung kênh

Được thực hiện định kỳ mỗi 30 giây:

Listing 15: Đồng bộ lên server (syncService.py)

```
1 for msg in messages:  
2     request_data = {  
3         "action": "send_message",  
4         "username": self.username,  
5         "channel_name": self.channel_name,  
6         "message": msg  
7     }
```

Listing 16: Đồng bộ xuống peer

```
1 response = channelRequest.handle_channel_request(json.dumps({  
2     "action": "get_channel_info",  
3     "channel_name": self.channel_name,
```

```
4     "username": self.username
5  })
```

Tham gia và chấp nhận yêu cầu vào channel

Người dùng có thể tham gia kênh public hoặc gửi yêu cầu vào kênh private. Chủ kênh có quyền duyệt/reject:

Listing 17: Approve user vào channel

```
1 request_data = {
2     "action": "approve_join_request",
3     "channel_name": self.current_channel,
4     "owner": self.username,
5     "target_user": username
6 }
```

Như vậy, hệ thống hỗ trợ đầy đủ chức năng kênh, lưu trữ, phân quyền, đồng bộ và phát trực tiếp, đảm bảo đáp ứng mục tiêu học phần về kết nối dữ liệu giữa các máy tính.

2.5 User Access

2.5.1 Lý thuyết

Hệ thống hỗ trợ một cơ chế truy cập người dùng thông minh, đảm bảo các chức năng chính sau:

- **User Fetch:** Khi người dùng chuyển từ trạng thái *offline* sang *online*, hệ thống cần tự động tải lại nội dung mới từ các channel mà họ đã tham gia.
- **User Notification:** Khi có tin nhắn mới hoặc hoạt động trên channel, người dùng nên được thông báo. Một giải pháp đơn giản là cơ chế *polling* theo chu kỳ và cập nhật giao diện nếu phát hiện thay đổi.

- **User List:** Mỗi instance của ứng dụng có thể đại diện cho một phiên đăng nhập riêng. Do đó, danh sách người dùng được định danh bằng bộ ba: (IP, Port, Username, Session ID).

2.5.2 Hiện thực hệ thống

Cơ chế tự động fetch dữ liệu khi online

Mỗi khi người dùng chọn một channel, một tiến trình đồng bộ sẽ được khởi tạo nếu chưa có:

Listing 18: Khởi động SyncManager (Home.py)

```
1 if not self.sync_manager:  
2     self.sync_manager = SyncManager(self.username, self.current_channel)  
3     self.sync_manager.start_periodic_sync(interval=30)
```

syncService.py định nghĩa phương thức sync_down để fetch nội dung từ server:

Listing 19: Lấy dữ liệu mới từ server

```
1 response = channelRequest.handle_channel_request(json.dumps({  
2     "action": "get_channel_info",  
3     "channel_name": self.channel_name,  
4     "username": self.username  
5 })))
```

Cơ chế Notification đơn giản bằng Polling

Thread poll_loop trong Home.py liên tục kiểm tra nội dung channel:

Listing 20: Polling để phát hiện nội dung mới

```
1 if len(messages) > self.last_message_count:  
2     new_msgs = messages[self.last_message_count:]  
3     for msg in new_msgs:  
4         self.chat_display.append(f"{msg['sender']}: {msg['text']}")
```

Cơ chế này đóng vai trò như một thông báo đơn giản và cập nhật giao diện theo thời gian thực.

Quản lý danh sách người dùng và các phiên làm việc

Server lưu các session với thông tin IP, port và session ID:

Listing 21: Lấy danh sách người dùng (authService.py)

```
1 users = list(users_collection.find({}, {"username": 1, "sessions": 1, "_id": 0})  
    )
```

Giao diện gọi API để hiển thị user list:

Listing 22: Tải danh sách user (Home.py)

```
1 request_data = {"action": "get_all_users"}  
2 response = json.loads(channelRequest.handle_channel_request(json.dumps(  
    request_data)))
```

Trong đó mỗi session được định danh duy nhất bởi bộ:

(peer_ip, peer_port, username, session_id)

Thông qua các cơ chế trên, hệ thống đảm bảo trải nghiệm người dùng liền mạch và phản ứng kịp thời với các sự kiện trong mạng P2P.

2.6 System Log

2.6.1 Lý thuyết ghi log hệ thống

Trong một hệ thống truyền thông đa hướng như mô hình hybrid peer-to-peer và client-server, việc theo dõi quá trình khởi tạo kết nối và truyền dữ liệu là cực kỳ quan trọng để:

- Hỗ trợ kiểm tra, gỡ lỗi trong quá trình phát triển.
- Đánh giá tính ổn định và khả năng mở rộng của hệ thống.
- Ghi nhận lịch sử truy cập và sự kiện giữa các peer hoặc với server.

Do đó, hệ thống cần xây dựng một cơ chế **ghi log toàn diện** cho mỗi sự kiện kết nối hoặc truyền dữ liệu. File log được lưu dưới dạng văn bản ASCII thuần để đơn giản hoá xử lý và đánh giá.

Yêu cầu kỹ thuật:

- Ghi nhận log khi có peer mới kết nối hoặc ngắt kết nối.
- Ghi lại các sự kiện gửi/nhận dữ liệu (tin nhắn, livestream, thông báo).
- File log có định dạng dễ đọc và có thể xoá/ghi đè khi vượt quá kích thước.

2.6.2 Hiện thực ghi log trong hệ thống

Cấu hình logging

Việc ghi log được thiết lập tập trung tại `Run.py`, sử dụng thư viện chuẩn logging của Python:

Listing 23: Cấu hình ghi log vào file và console

```
1 logger = logging.getLogger()
2 logger.setLevel(logging.INFO)
3 formatter = logging.Formatter('%(asctime)s %(levelname)s: %(message)s')
4
5 file_handler = logging.FileHandler('system.log', mode='a', encoding='utf-8')
6 file_handler.setFormatter(formatter)
7 logger.addHandler(file_handler)
8
9 console_handler = logging.StreamHandler()
10 console_handler.setFormatter(formatter)
11 logger.addHandler(console_handler)
```

File log mặc định được ghi vào `system.log` cùng thư mục thực thi và lưu mọi sự kiện hoạt động chính của hệ thống.

Ghi nhận kết nối Peer-to-Peer

Khi một peer mới khởi tạo server TCP để tiếp nhận kết nối, thông tin được ghi lại:

Listing 24: Ghi log khởi tạo server (user.py)

```
1 logging.info("P2P SERVER listening on %s:%s", self.ip, self.port)
```

Khi peer kết nối hoặc nhận dữ liệu từ peer khác:

Listing 25: Log dữ liệu nhận qua P2P

```
1 logging.info("[P2P MESSAGE] %s -> %s: %s", sender, self.name, text)
```

Ghi nhận gửi tin tracker và trạng thái

Trong quá trình gửi tin qua tracker:

Listing 26: Log gửi tin tracker

```
1 logging.info("Sent chat message via tracker: %s", message)
```

Khi peer rời mạng:

Listing 27: Ghi log LEAVE tracker

```
1 logging.info("Tracker response on LEAVE: %s", response_data.get('message', 'No response'))
```

Ghi nhận quá trình đồng bộ kênh

Cơ chế đồng bộ channel với server trung tâm cũng được ghi lại:

Listing 28: Log đồng bộ tin nhắn (syncService.py)

```
1 logging.info("[SYNC] Syncing %d local messages for channel '%s'", len(messages), self.channel_name)
```

Hoặc khi có lỗi:

```
1 logging.error("[SYNC] Error syncing local messages: %s", e)
```

Cơ chế ghi log giúp đảm bảo hệ thống minh bạch, dễ truy vết lỗi và hỗ trợ kiểm tra tính chính xác trong truyền dữ liệu mạng đa chiều.

2.7 Synchronization giữa Channel-hosting và Server trung tâm

2.7.1 Lý thuyết đồng bộ dữ liệu

Trong kiến trúc hybrid, một người dùng có thể là chủ sở hữu của một hoặc nhiều channel (**channel-hosting**). Khi peer hosting hoạt động (online), dữ liệu được lưu trữ tại local; khi offline, hệ thống cần sử dụng bản sao tại **server trung tâm** để đảm bảo tính liên tục trong truy cập và đồng bộ.

Mục tiêu của cơ chế đồng bộ:

- Đảm bảo mọi nội dung mới (tin nhắn) được cập nhật cả tại peer và server.
- Cho phép người dùng mới hoặc vừa online có thể truy xuất đầy đủ nội dung.
- Tránh mất mát dữ liệu khi peer tạo nội dung ở chế độ offline.
- Đồng bộ hai chiều: từ peer lên server và từ server về peer.

2.7.2 Tình huống và chiến lược đồng bộ

1. Khi peer host đang offline:

- Các joined peer sẽ truy xuất nội dung từ bản sao backup tại server trung tâm.
- Nếu người dùng tạo nội dung mới, nội dung này được ghi nhận tại server.
- Khi host online trở lại, nội dung sẽ được **sync từ server về local**.

2. Khi peer host online:

- Mọi cập nhật sẽ được ghi đồng thời tại peer host và server trung tâm.
- Các peer khác fetch dữ liệu từ peer host (truy cập nhanh hơn).

3. Khi joined peer vừa online:

- Kiểm tra timestamp các tin nhắn.
- Nếu có nội dung mới hơn local, sẽ thực hiện đồng bộ `sync_down()`.

4. Livestream period:

- Nội dung không đồng bộ lên server vì tính thời gian thực, chỉ gửi qua UDP trực tiếp.
- Một bản sao có thể lưu tại server khi host ngắt kết nối.

2.7.3 Hiện thực kỹ thuật trong mã nguồn

Cơ chế đồng bộ hai chiều được hiện thực bằng class `SyncManager` trong `syncService.py`, khởi tạo tại `Home.py` khi người dùng chọn channel.

Đồng bộ từ local lên server – `sync_up()`

Gửi các tin nhắn được lưu trong file cục bộ khi offline:

Listing 29: Đồng bộ từ local lên server

```
1 for msg in messages:
2     request_data = {
3         "action": "send_message",
4         "username": self.username,
5         "channel_name": self.channel_name,
6         "message": msg
7     }
8     response = channelRequest.handle_channel_request(json.dumps(request_data))
```

Sau khi gửi thành công, file cục bộ được xóa để tránh trùng lặp:

```
1 os.remove(self.local_sync_file)
```

Đồng bộ từ server về local – `sync_down()`

So sánh timestamp để xác định tin nhắn mới:

Listing 30: Đồng bộ từ server xuống peer

```
1 response = channelRequest.handle_channel_request(json.dumps({
2     "action": "get_channel_info",
3     "channel_name": self.channel_name,
4     "username": self.username
5 })))
6 messages = response.get("messages", [])
7 new_messages = [msg for msg in messages if msg["timestamp"] > self.
    last_sync_time]
```

Giao diện sẽ được cập nhật nếu phát hiện tin nhắn mới.

Khởi động đồng bộ định kỳ

Được thực hiện mỗi 30 giây:

```
1 self.sync_manager.start_periodic_sync(interval=30)
```

Thread này thực hiện đồng bộ liên tục trong suốt thời gian channel còn được mở.

2.8 Livestream Functionality

2.8.1 Lý thuyết truyền hình ảnh thời gian thực qua mạng P2P

Trong hệ thống, chức năng livestream là một phần mở rộng quan trọng, mô phỏng hành vi truyền phát video trong các ứng dụng như Discord. Mục tiêu là cho phép người dùng chủ kênh (host) truyền hình ảnh webcam của mình đến các người xem khác trong cùng channel theo thời gian thực, với độ trễ thấp.

Yêu cầu kỹ thuật:

- Sử dụng mô hình **Peer-to-Peer (P2P)** để truyền trực tiếp dữ liệu livestream từ host đến các peer khác.
- Áp dụng giao thức **UDP** để giảm độ trễ và tối ưu tốc độ truyền ảnh.
- Chuẩn hoá dữ liệu hình ảnh (frame) thành chuỗi văn bản để truyền qua socket.
- Hạn chế kích thước gói tin, giảm tần số gửi frame và chất lượng nén vừa đủ.

2.8.2 Các công cụ và thư viện sử dụng

- **OpenCV** (cv2): để truy cập webcam, xử lý khung hình và mã hoá ảnh.
- **Base64**: mã hoá dữ liệu nhị phân của ảnh JPEG thành chuỗi văn bản để gửi qua UDP.
- **socket (UDP)**: truyền dữ liệu ảnh đến các peer trong mạng.
- **NumPy**: giải mã byte ảnh để hiển thị.

2.8.3 Hiện thực quá trình livestream

Bên phía host (người livestream)

Tại peer chủ kênh, webcam được truy cập qua OpenCV. Mỗi khung hình được nén JPEG, mã hoá base64 và gửi đến tất cả các peer bằng UDP broadcast:

Listing 31: Phát livestream từ webcam (user.py)

```
1 ret, frame = cap.read()
2 ret, buffer = cv2.imencode('.jpg', frame, [int(cv2.IMWRITE_JPEG_QUALITY), 50])
3 jpg_as_text = base64.b64encode(buffer).decode('utf-8')
4
5 self.send_udp_broadcast(jpg_as_text, msg_type="livestream", channel=self.
    livestream_channel)
```

Gửi đi bằng UDP:

Listing 32: Gửi ảnh bằng UDP

```
1 udp_socket.sendto(data.encode('utf-8'), (target_ip, target_udp_port))
```

Bên phía viewer (người xem)

Mỗi peer chạy một tiến trình `udp_listener` chờ dữ liệu UDP. Khi nhận được chuỗi base64, dữ liệu được giải mã và hiển thị bằng OpenCV:

Listing 33: Nhận và hiển thị livestream (user.py)

```
1 img_bytes = base64.b64decode(frame_data)
2 np_arr = np.frombuffer(img_bytes, np.uint8)
3 frame = cv2.imdecode(np_arr, cv2.IMREAD_COLOR)
4 cv2.imshow(f"UDP Livestream from {sender}", frame)
```

Chỉ những peer đang xem đúng channel mới hiển thị được nội dung livestream:

```
1 if not self.active_channel or self.active_channel != msg_channel:
2     continue
```

Tích hợp với UI và quyền truy cập

Người dùng có thể bắt đầu/đóng livestream từ giao diện:

Listing 34: Giao diện Start/Stop Livestream (Home.py)

```
1 if not self.user_peer.is_livestreaming:
2     threading.Thread(target=self.user_peer.start_livestream, daemon=True).start
3     ()
4 else:
5     self.user_peer.stop_livestream()
```

Chỉ người dùng là chủ channel mới có quyền phát livestream, các peer khác chỉ được phép xem:

```
1 if self.current_channel_info["owner"] != self.username:
2     self.chat_display.append("[INFO] Only owner can start livestream.")
```

3 Các bước thực hiện hệ thống (System Workflows)

Để minh họa rõ cách hoạt động của hệ thống chat theo mô hình hybrid, dưới đây là các tình huống tiêu biểu. Mỗi bước được ghi chú rõ hàm hoặc module nào được gọi khi hành động xảy ra.

3.1 Tình huống 1: Đăng nhập hoặc truy cập với tư cách khách

Ngữ cảnh: Người dùng mở ứng dụng lần đầu và muốn tham gia hệ thống.

1. Mở `Run.py` khởi động giao diện.
2. Hiện thị `LoginRegisterUI` từ `Login.py`.
3. Nếu người dùng đăng nhập:
 - Hàm: `login()` → gửi request action: `login`.
 - File xử lý: `authRequest.py` → `authController.py` → `authService.py` → `login_user()`.
 - Tracker gọi: `connect_to_tracker()` trong `user.py`.
4. Nếu người dùng chọn `Login as Viewer`:
 - Hàm: `login_as_viewer()` (`Login.py`).
 - Không xác thực, chỉ tạo phiên với tên.
5. Chuyển đến giao diện chính `DiscordUI` (`Home.py`).

3.2 Tình huống 2: Tạo một kênh mới

Ngữ cảnh: Người dùng muốn tạo một kênh trao đổi riêng.

1. Nhấn nút + `Create Channel`.

2. Lựa chọn chế độ private hay public.
3. Nhập tên và chọn tính năng cho phép visitor xem không.
4. Gọi `add_channel()` → `handle_channel_request()` với `action: create_channel`.
5. Controller: `channelController.py` → `create_channel_controller()`.
6. Service: `create_channel()` trong `channelService.py`.
7. Channel được lưu trong MongoDB và cập nhật UI.

3.3 Tình huống 3: Tham gia một channel đã tồn tại

Ngữ cảnh: Người dùng muốn tham gia channel đang hoạt động.

1. Nhấn vào tên channel trong `channel_list`.
2. Hàm: `handle_channel_clicked()` (`Home.py`) → Gọi `get_channel_info`.
3. Nếu chưa tham gia, nhấn `Join Channel` → gọi `join_channel()`.
4. Controller: `join_channel_controller()` → Service: `join_channel()`.
5. Nếu là kênh riêng (private), yêu cầu sẽ chờ duyệt trong `join_requests`.

3.4 Tình huống 4: Gửi tin nhắn trong channel

Ngữ cảnh: Người dùng muốn gửi nội dung chat.

1. Nhập nội dung, nhấn nút `Send` hoặc `Enter`.
2. Gọi hàm: `send_message()` → `send_message_p2p_api()` (`Home.py`).
3. Gửi request với `action: send_message`.
4. Controller: `send_message_controller()` → `send_message_p2p()` (`channelService.py`).

5.
 - Nếu host channel online, thì tin nhắn từ các peer khác sẽ được gửi đến p2p server của host channel sau đó đồng bộ lên MongoDB, bên cạnh đó các peer khác nhận tin nhắn từ host channel.
 - Nếu host channel offline, thì tin nhắn từ các peer sẽ được gửi thẳng lên MongoDB và khi chủ kênh online thì sẽ đồng bộ với MongoDB (tức là lấy các tin nhắn mới từ MongoDB xuống file cục bộ)

3.5 Tình huống 5: Bắt đầu và xem livestream

Ngữ cảnh: Chủ kênh phát webcam, các thành viên khác xem.

1. Nhấn Start Livestream từ UI → gọi `toggle_livestream()`.
2. Gọi: `start_livestream()` trong `user.py`.
3. Webcam được mở bằng OpenCV: `cv2.VideoCapture`.
4. Mỗi frame được:
 - Nén bằng JPEG: `cv2.imencode`.
 - Mã hóa base64: `base64.b64encode`.
 - Gửi qua UDP: `send_udp_broadcast()`.
5. Các peer khác nhận tại: `udp_listener()` → giải mã và hiển thị bằng `cv2.imshow`.
6. Khi nhấn lại Stop Livestream → gọi `stop_livestream()`.

3.6 Tình huống 6: Người dùng trở lại sau khi offline

Ngữ cảnh: Một peer chuyển từ offline sang online.

1. Người dùng mở lại ứng dụng và đăng nhập.
2. Khi đó tất cả tin nhắn mới khi người dùng offline (người dùng là chủ kênh) sẽ đồng bộ từ MongoDB về file cục bộ `login()` → `get_hosted_channels()` → *Synchronous*

3. Trong `handle_channel_clicked()`:

- Trong trường hợp là người dùng là chủ các kênh thì sẽ lập tức đồng bộ các tin nhắn mới từ MongoDB tức là nếu có tin nhắn mới trong thời gian chủ kênh offline thì sẽ ghi các tin nhắn đó vào file cục bộ.
 - Trong trường hợp là người dùng không phải chủ kênh thì sẽ nhận tin nhắn và gửi tin nhắn dựa trên việc chủ kênh đã online tức là gửi và nhận thông qua p2p server của host channel đó.
4. Khi chủ kênh online thì mọi thay đổi tin nhắn trong kênh sẽ được ghi vào file cục bộ của họ (*`poll_loop()` $\rightarrow$ `get_channel_info()` $\rightarrow$ *Synchronous**).
5. Lúc này, tất cả các hoạt động nhắn tin và livestream sẽ thực hiện như tình huống 4 và 5.

3.7 Tình huống 7: Người dùng đăng xuất và offline

Ngữ cảnh: Một peer chuyển từ online sang offline.

1. Người dùng đăng xuất khỏi tài khoản.

2. Trong `logout`:

- Trong trường hợp là người dùng là chủ các kênh thì sẽ lập tức đồng bộ các tin nhắn chưa đồng bộ từ tất cả các file cục bộ lên MongoDB.
 - Trong trường hợp là người dùng không phải chủ kênh thì sẽ nhận tin nhắn và gửi tin nhắn dựa trên việc chủ kênh đã offline tức là gửi và nhận thông qua tracker server.
3. Khi chủ kênh chuyển từ online sang offline sẽ được ghi vào file cục bộ của họ (*`logout()` $\rightarrow$ *Synchronous**).
4. Lúc này, tất cả các hoạt động nhắn tin và livestream sẽ thực hiện như tình huống 4 và 5.



4 Nguồn dữ liệu và nguồn code

Source Code.

Tài liệu